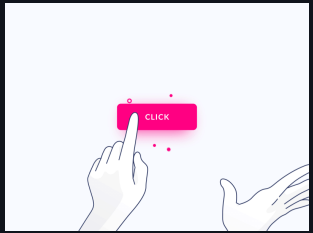
Interaction

Mitsiu Alejandro Carreño Sarabia



Agenda

- Recap
- Elm arquitecture
- Model (State)
- View
- Update
  - Msg
- View onClick
- Wiring everything up
- Homework

Recap

Before I forget

Recap

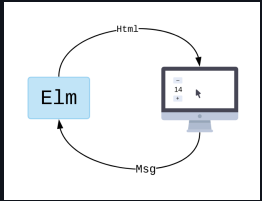
Our next tests:

- 2nd Partial = 30%
  - Class exercise / Homework = 10%
  - Theorical evaluation = 40%
  - Practical evaluation (paper based, NO computer!) = 50%

Elm arquitecture

Elm arquitecture

The basic pattern looks something like this:

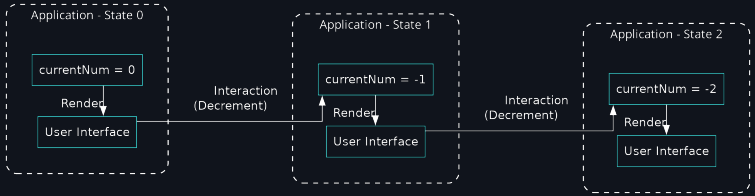


The Elm program produces HTML to show on screen, and then the computer sends back messages of what is going on. "They clicked a button!"

- An application starts with an initial state and presents that state to the user through UI.
- The user takes some action through a UI element that modifies the inital state.
- The new state is then presented back to the user for more actions.

Elm arquitecture

The basic pattern looks something like this:



What happens within the Elm program though? It always breaks into three parts:

- Model – the state of your application
- View – a way to turn your state into HTML
- Update – a way to update your state based on messages

▒▒▒ Model (State) - the state of your application

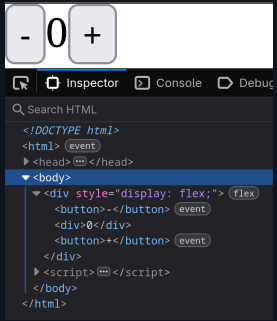
Our application is really simple we want to keep track of a single Integer in a property currentNum inside a record:

```
type alias Model =
    { currentNum : Int }


init : Model
init =
    { currentNum = 0 }
```

During the whiteboard explanation I named it as model, but it's clearer to name it init, because it's our initial state

View - A way to turn your state into HTML




```elm
view : Html.Html Msg
view =
    Html.div
        [ Html.Attributes.
style "display" "flex" ]
        [ Html.button
            [ ]
            [ Html.text "-" ]
        , Html.div
            []
            [ Html.text "0" ]
        , Html.button
            [ ]
            [ Html.text "+" ]
        ]
```

▓▓▓ View - A way to turn your state into HTML

We can turn our variable into a function based on our current state:

```
view : Html.Html Msg
view =
    ...
            [ Html.text "0" ]
```

```
view : Model -> Html.Html Msg
view model =
    ...
            [ Html.text (
String.fromInt (.currentNum
model)) ]
```

▓▓▓ View - A way to turn your state into HTML

Here's the complete view function

```
view : Model -> Html.Html Msg
view model =
    Html.div
        [ Html.Attributes.style "display" "flex" ]
        [ Html.button [ ] [ Html.text "-" ]
        , Html.div
            []
            [ Html.text (String.fromInt (.currentNum model
)) ]
        , Html.button [ ] [ Html.text "+" ]
        ]
```

▒▒▒ Update - A way to update your state based on messages

Our update function will take our current model and produce the next version of our model, but it also need's to know what interaction happened.

```
update : _____ -> Model -> Model
update ___ model =
    case ___ of
        Increment ->
            { model | currentNum = .currentNum model + 1 }

        Decrement ->
            { model | currentNum = .currentNum model - 1 }
```

▒▒ Update - A way to update your state based on messages

▒▒ Msg - Interactions

In elm we name our interactions as Msg (messages)

Our application has two possible interactions:

- Increment (click the + button)
- Decrement (click the - button)

So we build a custom type that exactly fit's our application:

```
type Msg
    = Increment
    | Decrement
```

Update - A way to update your state based on messages

```
type Msg
    = Increment
    | Decrement

update : Msg -> Model -> Model
update msg model =
    case msg of
        Increment ->
            { model | currentNum = .currentNum model + 1 }
        Decrement ->
            { model | currentNum = .currentNum model - 1 }
```


View onClick

View onClick

Let's revisit our view function:

```
view : Model -> Html.Html Msg
view model =
    Html.div
        [ Html.Attributes.style "display" "flex" ]
        [ Html.button [ ] [ Html.text "-" ]
        , Html.div
            []
            [ Html.text (String.fromInt (.currentNum model
)) ]
        , Html.button [ ] [ Html.text "+" ]
        ]
```

View onClick

It has a way to translate our state into HTML but it cannot send interactivity Messages

```elm
view : Model -> Html.Html Msg
view model =
    Html.div
        [ Html.Attributes.style "display" "flex" ]
        [ Html.button [ ] [ Html.text "-" ]
        , Html.div
            []
            [ Html.text (String.fromInt (.currentNum model)) ]
        , Html.button [ ] [ Html.text "+" ]
        ]
```

View onClick

Elm has Events like onClick : msg -> Attribute msg

```
1  import Html.Events
2
3  view model =
4      Html.div
5          [ Html.Attributes.style "display" "flex" ]
6          [ Html.button
7              [ Html.Events.onClick Decrement ]
8              [ Html.text "-" ]
9          , Html.div
10             []
11             [ Html.text (String.fromInt (.currentNum
   model)) ]
12         , Html.button
13             [ Html.Events.onClick Increment ]
14             [ Html.text "+" ]
15         ]
```


Wiring everything up

Wiring everything up

```
module Main exposing (..)

import Browser
import Html
import Html.Attributes
import Html.Events

main =
    Browser.sandbox
        { init = init
        , update = update
        , view = view
        }

type alias Model =
    { currentNum : Int }
```

```
init : Model
init =
    { currentNum = 0 }

type Msg
    = Increment
    | Decrement
```

Wiring everything up

```
update : Msg -> Model -> Model
update msg model =
    case msg of
        Increment ->
            { model | currentNum = .currentNum model + 1 }

        Decrement ->
            { model | currentNum = .currentNum model - 1 }
```

Wiring everything up

```
view : Model -> Html.Html Msg
view model =
    Html.div
        [ Html.Attributes.style "display" "flex" ]
        [ Html.button
            [ Html.Events.onClick Decrement ]
            [ Html.text "-" ]
        , Html.div
            []
            [ Html.text (String.fromInt (.currentNum model
)) ]
        , Html.button
            [ Html.Events.onClick Increment ]
            [ Html.text "+" ]
        ]
```

Wiring everything up